PROJECT REPORT: INTERACTIVE GEOSPATIAL DASHBOARD FOR SRI LANKA POPULATION DENSITY

Coursework Title: Interactive Dashboards Using Python

Index Number: COHNDDS251P-002

Name: J G T Ridmika

Selected Pool: Pool 2 - Geospatial Visualization with Bokeh

1. Introduction

1.1 Overview

The analysis of geospatial data enables the derivation of insights related to demographic, economic, and environmental factors. For Sri Lanka, it is critical to present an informative graphical view of the population density at a district and province levels for effective regional development and resource allocation.

This project aims to design and develop an interactive Web App Dashboard using the Python library Bokeh that will display population density for each district in Sri Lanka for the year 2024.

The 2024 dataset was chosen because the updates for 2025 and 2026 are not available on the official DCS statistical portal (www.statistics.gov.lk). Therefore, the 2024 data is relevant and up-to-date regarding geospatial density.

1.2 Objectives

- To integrate data containing Latitude & Longitude with the web mercator projection coordinate system.
- To implement advanced interactivity including hover tooltips, categorical filters, density range slider, and visual zoom.
- To deploy the Web Dashboard on cloud servers using the Bokeh curdoc model for visualization in web browsers.

2. Dataset & Preprocessing

2.1 Data Sources

The project uses the following dataset:

1. **District_Locations.csv**: This file contains information about the geographical location (latitude and longitude) of the 25 administrative districts in Sri Lanka. https://www.kaggle.com/datasets/liewyousheng/geolocation

2. **Demographic Datasets (2024 Estimated Data):** This file contains the 2024 estimated population, area in square kilometers, and province for each district. https://www.statistics.gov.lk/Population/StaticalInformation/CPH2024

2.2 Data Preprocessing Workflow

The following activities were carried out to prepare the data for analysis:

1. **Coordinate conversion:** Latitude & Longitude values (WGS84 GPS standard) were converted to web mercator (x,y meters) using the following formula:

```
def lat_lon_to_mercator(lat, lon):
    r_major = 6378137.0
    x = r_major * np.radians(lon)
    scale = x / lon
    y = (
        180.0
        / np.pi
        * np.log(np.tan(np.pi / 4.0 + lat * (np.pi / 180.0) / 2.0))
        * scale
    )
    return x, y
```

2. **Density calculation:** Population density (people/sq. km) was calculated for each district using the formula:

$$\text{Density} = \frac{\text{Population (2024)}}{\text{Area (sq. km)}}$$

3. **Dynamic Sizing:** The population density was used to determine the size (in pixels) of each scatter glyph using the following square-root scale:

```
df["size"] = np.sqrt(df["density"]) * 0.9 + 14
```

3. Bokeh Visualization Design

3.1 Map Creation & Styling

The following tools were used to create the dashboard:

- OSM tile source was used to add a bright theme to the map.
- LinearColorMapper with a Plasma256 color palette was used to indicate low/high population density.

3.2 Dashboard Widgets

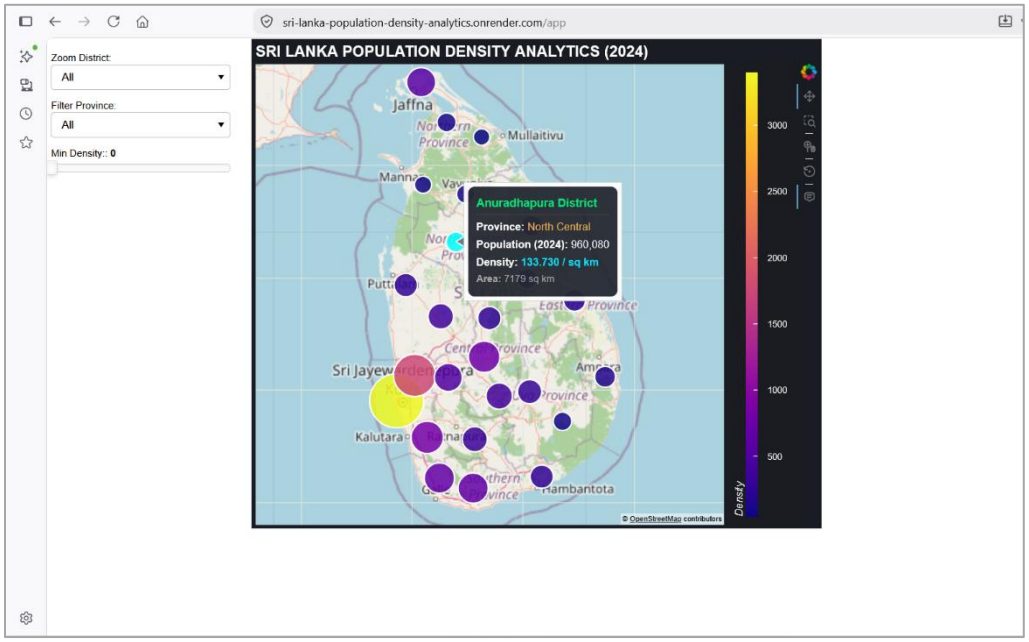
The following widgets were added to the dashboard:

- **Hover Tooltip:** This displays information about the hovered district in a card-style format including the district name, province, population, density, and area.
- **Province Select:** This filter displays only those districts that belong to the selected province
- **Min Density Slider:** This interactive widget hides lower-density districts as the slider value changes
- **Zoom District Control:** This select widget identifies the offsets (in meters) to zoom into a specified district.

```
def update_data(attr, old, new):
    filtered_df = df.copy()

    if select_province.value != "All":
        filtered_df = filtered_df[filtered_df["province"] == select_province.value]

    filtered_df = filtered_df[filtered_df["density"] >= slider_density.value]

    source.data = ColumnDataSource.from_df(filtered_df)
```

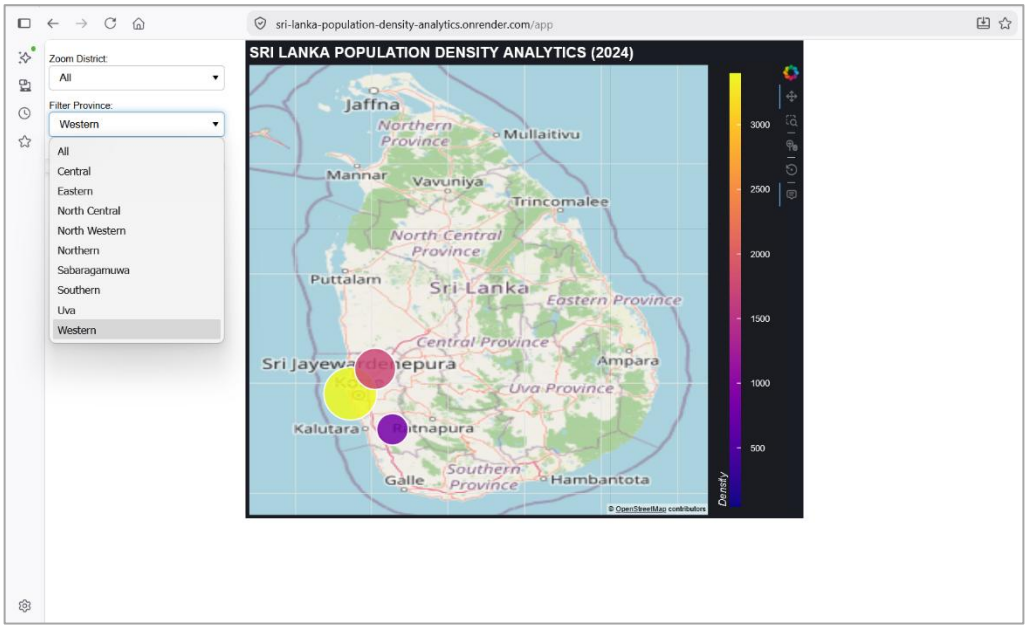
3.3 Dashboard Screenshots and Interactivity

Figure 1: Dashboard View and Hover Tooltip Functionality



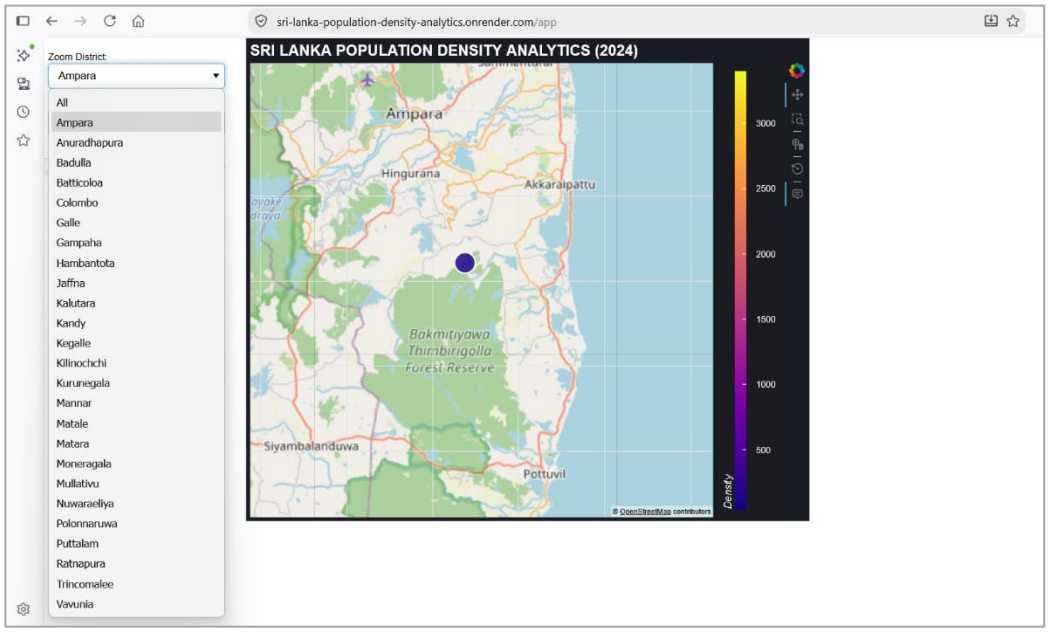
The figure above demonstrates the dark theme map UI, dynamic color scale, and custom HoverTool displaying district analytics upon mouse hover.

Figure 2: Province Filter Widget



The figure above shows the province filter widget.

Figure 3: Interactive Zoom Functionality



The figure above shows the interactive zoom functionality where the x/y offsets change automatically to focus on the selected district.

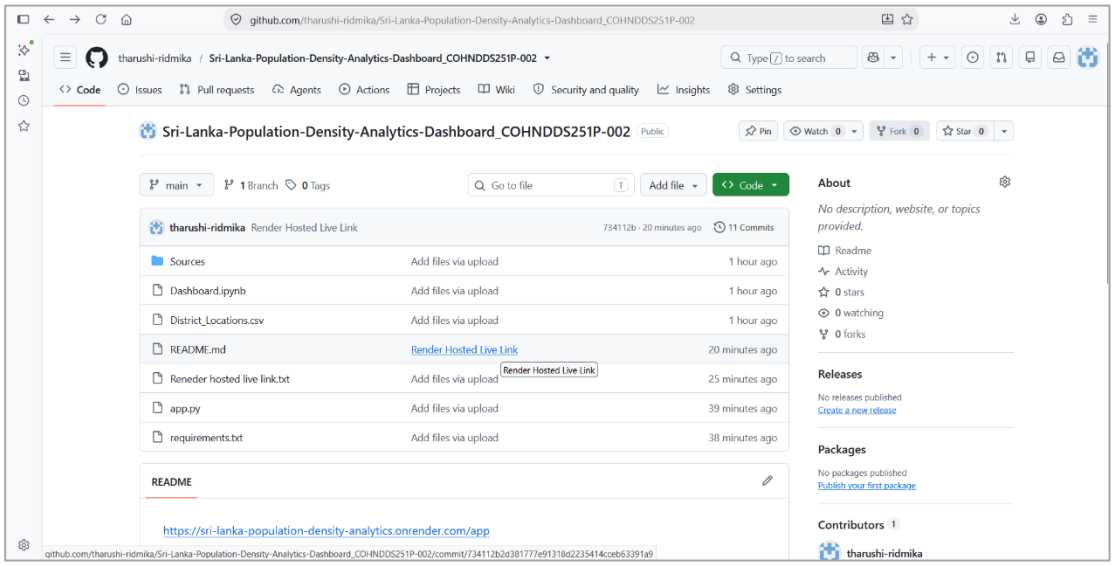
4. Hosting Procedure

4.1 Hosting Procedure Overview

To fulfill the requirement of deploying the app on cloud servers, a separate render procedure had to be designed. It involved developing a Render Web Service using the dashboard.ipynb notebook and deploying it on https://render.com/. The activities involved are described below:

- The app.py production script was developed using the curdoc().add_root(layout) method to enable pure Python callbacks.
- The dashboard.ipynb notebook was used for testing purposes with the CustomJS JavaScript callbacks.

Figure 4.1: Project Repository Structure on GitHub



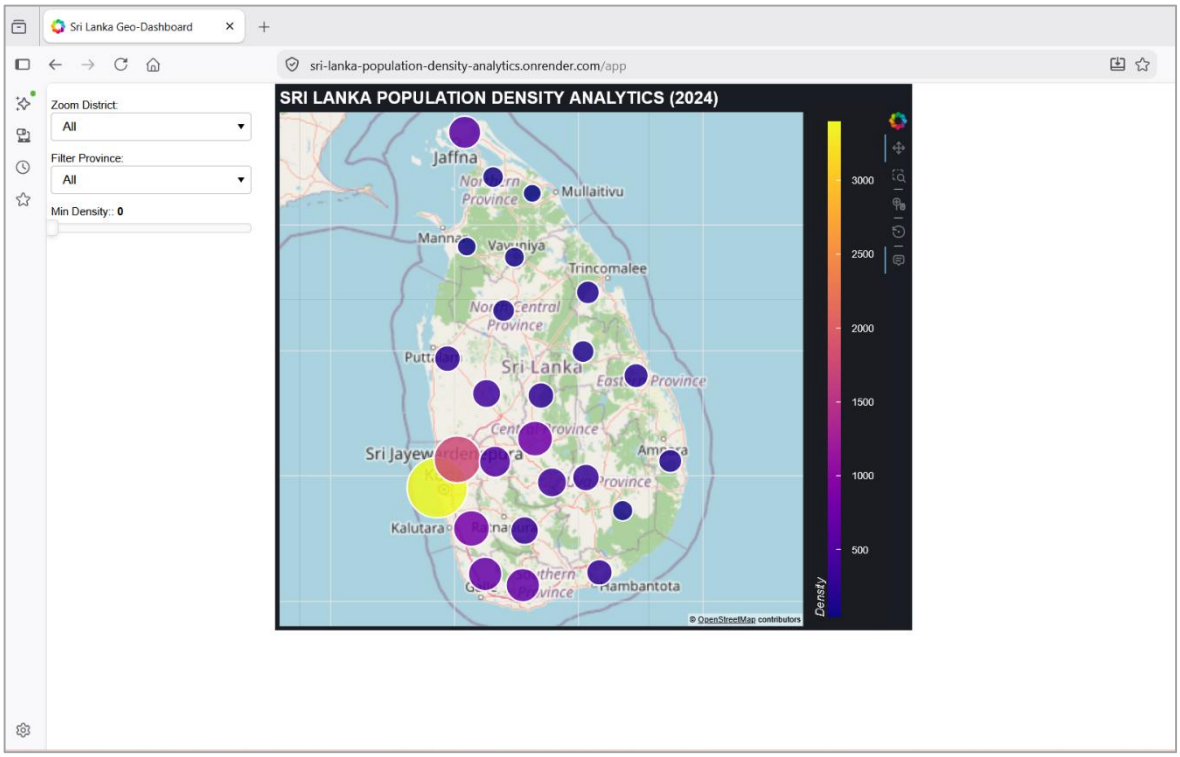
The figure above shows the GitHub repository web interface. It contains the production (app.py) and test (.ipynb) scripts, along with the District_Locations.csv file and requirements.txt file.

4.2 Render Hosting Step-by-Step Guide

The following steps were taken to host the web app on Render:

1. **Repository Setup (GitHub):** Structure pushed to public repository containing app.py, dashboard.ipynb, District_Locations.csv, and requirements.txt.
2. The requirements.txt file described the Python dependencies: bokeh pandas numpy.
    - bokeh
    - pandas
    - numpy
3. **Render Service Creation:**
    - **Service Type:** Web Service
    - **Build Command:** pip install -r requirements.txt
    - **Start Command:** bokeh serve app.py --port $PORT --allow-websocket-origin=*

Figure 4.2: Render Hosting Dashboard



The figure above shows the Render hosting dashboard

- **Live Interactive Application Link:**

  https://sri-lanka-population-density-analytics.onrender.com/app

5. Challenges and Improvements

5.1 Challenges

The following challenges were encountered while completing this project:

1. **Coordinate system conversion:** The conversion of standard latitude/longitude values to web mercator had to be done manually using a mathematical formula.
2. **Deploying on Render:** The transition from the notebook-based CustomJS JavaScript callbacks to Python curdoc() required significant code rewrites.
3. **Websocket origin errors:** The initial attempt to launch the app on Render resulted in connection errors that had to be resolved by specifying the –allow-websocket-origin= parameter.

5.2 Enhancements

The following enhancements can be considered for the future development of the dashboard project:

1. Using GeoJSON to define administrative boundaries (polygons) and display choropleth maps instead of point glyphs.

2. Adding a time-series slider to view the changes in population density over time.

6. Conclusion

The Sri Lanka Population Density Analytics dashboard was developed using the Python programming language and successfully deployed on the Render web server. The project met all course requirements by using the Bokeh framework to display an interactive map of Sri Lanka with the district population density for 2024.